

Condizioni su className in React

In React `className` deve sempre ricevere una stringa.

Quando una classe dipende da una condizione, meglio evitare che il risultato finale contenga valori come `false`.

Problema

```
<div className={'card ' + (isActive && 'active')} />
```

Se `isActive` e' `false`, la condizione ritorna `false`, questo genera un errore in console.

Oppure

```
<div className={`card ${isActive ? 'active' : false}`} />
```

Qui abbiamo un ternary operator, ma ritornando `false`, anche questo genera un errore in console.

Soluzione 1: ternary operator con stringa vuota

```
<div className={`card ${isActive ? 'active' : ''}`} />
```

Qui il risultato e' sempre una stringa:

- se `isActive` e' `true`: `'card active'`
- se `isActive` e' `false`: `'card '`

A chi interessa, esiste anche una soluzione piu' elegante, che permette di scrivere il codice senza ternary operator o template literal, vediamo come fare.

Soluzione 2: funzione cx

La funzione `cx` risolve entrambi i problemi: rimuove i valori falsy e permette di scrivere il codice senza ternary operator o template literal.

- Creare questa funzione in un file, per esempio `utils.js`.

```
export const cx = (...classes) => classes.filter(Boolean).join(' ');
```

- Importare la funzione `cx` nel file del componente

```
import { cx } from '../utils.js';
```

- Usare la funzione `cx` per concatenare classi CSS in modo condizionale

```
<div className={cx('card', isActive && 'active')} />
```

`cx` rimuove i valori falsy, quindi puoi usare `&&` senza ritrovarti `false` dentro `className`.

Nota sulla lezione

Giovedì Marco ha mostrato condizioni su `className` usando anche `&&`. In quel caso specifico forse andava bene, ma per prevenire errori che non possiamo prevedere (ritorno di valori falsy inaspettati), è più sicuro usare:

- ternary operator con stringa vuota `''`;
- oppure una utility come `cx`.

Tip

La utility `cx` è molto comune nei progetti React. Esistono anche librerie npm come `classnames` e `clsx` basate su questo concetto, con qualche funzionalità in più. La libreria più famosa di componenti React, ShadcnUI, la usa per gestire le classi CSS e TailwindCSS dei loro componenti, con una versione custom chiamata `cn`.

Solo a chi interessa: come funziona cx

- La funzione `cx` prende un numero indefinito di argomenti, che possono essere stringhe o valori falsy: `cx('a', 'b', false, 'c')`
- `...classes` è un rest parameter, che raccoglie tutti gli argomenti in un array chiamato `classes`.
- `classes.filter(Boolean)` rimuove tutti i valori falsy dall'array.
- `join(' ')` unisce tutti gli elementi dell'array in una stringa, separandoli con uno spazio. Es. `['a', 'b', 'c'].join(' ')` diventa `'a b c'`.
- Quindi quando la nostra condizione è `false`, la funzione ritorna solo le classi valide, senza errori in console. Esempio:

```
cx('card', isActive && 'active'); // se isActive e' false, ritorna 'card'  
cx('card', isActive && 'active'); // se isActive e' true, ritorna 'card  
active'
```

In pratica, `cx` serve a concatenare classi CSS in modo condizionale, mantenendo il codice piu' leggibile.